ĐỀ MẪU DỰ ÁN

Tên đề tài

Library Management System

Hệ thống quản lý thư viện bằng Java Swing MVC + Hibernate/JPA + MySQL

1. Mục tiêu dự án

Xây dựng một ứng dụng desktop quản lý thư viện cho trường học hoặc trung tâm đào tạo, hỗ trợ:

- quản lý sách
- quản lý tác giả, thể loại, nhà xuất bản
- quản lý độc giả
- quản lý phiếu mượn – trả
- tra cứu sách
- thống kê cơ bản

Dự án phải được phát triển theo mô hình:

- **Java Swing** cho giao diện
- **MVC** cho tổ chức mã nguồn
- **Hibernate/JPA** để ánh xạ đối tượng – quan hệ
- **MySQL** để lưu trữ dữ liệu
- **Maven** để quản lý thư viện
- áp dụng các nguyên tắc **SOLID**

2. Quy mô phù hợp

Dự án phù hợp cho:

- **2 sinh viên**: làm phần chức năng cốt lõi
- **3 sinh viên**: thêm báo cáo/thống kê, phân quyền, validate nâng cao

Thời lượng đề xuất:

- **2-3 tuần**

3. Yêu cầu công nghệ

3.1. Công nghệ bắt buộc

- Java 17 hoặc Java 21
- Java Swing
- Maven
- MySQL 8
- Hibernate ORM / JPA
- IntelliJ IDEA

3.2. Thư viện đề xuất

- hibernate-core
- jakarta.persistence-api
- mysql-connector-j
- lombok (không bắt buộc)
- slf4j-simple hoặc logback-classic

4. Bài toán nghiệp vụ

Thư viện cần một phần mềm giúp nhân viên:

1. Quản lý danh mục sách
2. Quản lý thông tin độc giả
3. Lập phiếu mượn sách
4. Ghi nhận trả sách
5. Kiểm tra tình trạng còn/hết sách
6. Tìm kiếm sách theo tên, tác giả, thể loại
7. Xem thống kê số lượt mượn

Ứng dụng chỉ cần chạy **desktop nội bộ**, không yêu cầu web.

5. Chức năng chính

5.1. Quản lý sách

- thêm sách mới
- cập nhật thông tin sách
- xóa sách
- xem danh sách sách
- tìm kiếm theo:
    - mã sách
    - tên sách
    - thể loại
    - tác giả
    - nhà xuất bản

5.2. Quản lý tác giả

- thêm tác giả
- sửa tác giả
- xóa tác giả
- xem danh sách tác giả

5.3. Quản lý thể loại

- thêm thể loại
- sửa thể loại
- xóa thể loại

5.4. Quản lý nhà xuất bản

- thêm/sửa/xóa nhà xuất bản

5.5. Quản lý độc giả

- thêm độc giả
- sửa thông tin độc giả
- khóa/mở độc giả
- tìm kiếm độc giả

5.6. Quản lý mượn sách

- lập phiếu mượn
- chọn độc giả
- chọn nhiều sách trong một phiếu
- kiểm tra số lượng còn
- tự động giảm tồn kho khi mượn thành công

5.7. Quản lý trả sách

- trả từng cuốn hoặc toàn bộ phiếu
- cập nhật ngày trả
- cập nhật tình trạng sách
- tăng tồn kho khi trả thành công

5.8. Thống kê

- số sách hiện có
- số độc giả đang hoạt động
- số phiếu mượn chưa trả
- top sách được mượn nhiều

6. Yêu cầu phi chức năng

- Giao diện dễ dùng, bố cục rõ ràng
- Tổ chức mã theo chuẩn MVC
- Có validate dữ liệu đầu vào
- Có xử lý lỗi cơ bản
- Không viết toàn bộ logic trong JFrame
- Không viết SQL thủ công cho toàn bộ dự án nếu đã dùng JPA
- Tách riêng:

- giao diện
- nghiệp vụ
- truy cập dữ liệu

7. Kiến trúc đề xuất

7.1. Mô hình tổng quát

- **View**: JFrame, JDialog, JPanel
- **Controller**: tiếp nhận sự kiện từ giao diện, gọi service
- **Model/Entity**: các lớp JPA entity
- **Repository/DAO**: thao tác dữ liệu qua EntityManager/Hibernate
- **Service**: xử lý nghiệp vụ

7.2. Package đề xuất

```
src/main/java
└── com.example.library
    ├── config
    │   └── JpaUtil.java
    ├── model
    │   ├── entity
    │   └── dto
    ├── repository
    ├── service
    ├── service.impl
    ├── view
    │   ├── main
    │   ├── book
    │   ├── reader
    │   └── loan
    ├── controller
    ├── util
    └── App.java
```

8. Thiết kế dữ liệu và ERD đề xuất

8.1. Các bảng chính

1) categories

- id
- name
- description

2) publishers

- id
- name
- address
- phone

3) authors

- id
- full_name
- email
- bio

4) books

- id
- isbn
- title
- publish_year
- price
- quantity
- available_quantity

- category_id
- publisher_id

5) book_authors

- book_id
- author_id

6) readers

- id
- reader_code
- full_name
- gender
- phone
- email
- address
- status

7) staffs

- id
- staff_code
- full_name
- username
- password
- role

8) loans

- id
- loan_code
- reader_id
- staff_id
- loan_date
- due_date
- status

9) loan_details

- id
- loan_id
- book_id
- quantity
- returned_quantity
- return_date
- note

9. Entity classes phù hợp với ERD

Đây là phần rất quan trọng vì sinh viên được yêu cầu **Class Diagram phải khớp với ERD**.

9.1. Danh sách entity

- Category
- Publisher
- Author
- Book
- Reader
- Staff
- Loan
- LoanDetail

9.2. Quan hệ JPA đề xuất

Category – Book

- One-to-Many
  Một thể loại có nhiều sách

Publisher – Book

- One-to-Many
  Một nhà xuất bản có nhiều sách

Book – Author

- Many-to-Many
  Một sách có thể có nhiều tác giả, một tác giả có thể viết nhiều sách

Reader – Loan

- One-to-Many
  Một độc giả có nhiều phiếu mượn

Staff – Loan

- One-to-Many
  Một nhân viên lập nhiều phiếu mượn

Loan – LoanDetail

- One-to-Many
  Một phiếu mượn có nhiều dòng chi tiết

Book – LoanDetail

- One-to-Many
  Một sách có thể xuất hiện trong nhiều dòng chi tiết phiếu mượn

10. Gợi ý class diagram logic

10.1. Nhóm entity

- Category
- Publisher
- Author
- Book
- Reader
- Staff
- Loan
- LoanDetail

10.2. Nhóm nghiệp vụ

- BookRepository
- ReaderRepository
- LoanRepository
- BookService
- ReaderService
- LoanService
- BookController
- ReaderController
- LoanController

10.3. Quan hệ phụ trợ

- JpaUtil
- ValidatorUtil
- MessageBoxUtil

11. Use Case Diagram cần có

11.1. Actors

- **Staff**
- **Admin**
  (Admin có thể kế thừa Staff nếu muốn mở rộng)

11.2. Use cases bắt buộc

Staff

- Đăng nhập
- Quản lý sách
- Quản lý tác giả
- Quản lý thể loại
- Quản lý nhà xuất bản
- Quản lý độc giả
- Lập phiếu mượn
- Ghi nhận trả sách
- Tìm kiếm sách
- Xem thống kê

Admin

- Quản lý nhân viên
- Xem toàn bộ báo cáo

11.3. Quan hệ include/extend gợi ý

- Lập phiếu mượn **include** Kiểm tra số lượng sách
- Ghi nhận trả sách **include** Cập nhật tồn kho
- Quản lý sách có thể bao gồm:
    - thêm sách
    - sửa sách
    - xóa sách
    - tìm kiếm sách

12. Activity Diagram cần có

Nhóm phải vẽ tối thiểu các activity sau:

12.1. Activity: Lập phiếu mượn

Luồng gợi ý:

1. Nhân viên chọn chức năng lập phiếu mượn
2. Tìm độc giả
3. Chọn sách
4. Kiểm tra số lượng còn
5. Nếu hợp lệ → tạo phiếu mượn
6. Tạo chi tiết phiếu mượn
7. Giảm số lượng khả dụng
8. Thông báo thành công

12.2. Activity: Trả sách

1. Chọn phiếu mượn
2. Chọn sách trả
3. Cập nhật returnedQuantity
4. Cập nhật ngày trả
5. Tăng availableQuantity
6. Nếu trả hết → cập nhật trạng thái phiếu

12.3. Activity: Thêm sách

1. Nhập dữ liệu sách
2. Kiểm tra rỗng
3. Kiểm tra ISBN trùng
4. Lưu DB
5. Reload bảng dữ liệu

13. Sequence Diagram cần có

Tối thiểu nên có 3 sequence diagram.

13.1. Sequence: Thêm sách

Staff → BookView → BookController → BookService → BookRepository → DB

Luồng:

1. Staff nhập dữ liệu trên form
2. View gửi yêu cầu tới Controller
3. Controller gọi Service validate
4. Service gọi Repository lưu dữ liệu
5. Repository dùng JPA persist
6. Trả kết quả về View

13.2. Sequence: Lập phiếu mượn

Staff → LoanView → LoanController → LoanService → ReaderRepository / BookRepository / LoanRepository → DB

Điểm cần thể hiện:

- kiểm tra độc giả tồn tại
- kiểm tra sách còn
- tạo Loan
- tạo LoanDetail
- cập nhật Book.availableQuantity

13.3. Sequence: Trả sách

Staff → ReturnView → LoanController → LoanService → LoanRepository / BookRepository → DB

Điểm cần thể hiện:

- lấy chi tiết phiếu
- cập nhật số lượng đã trả
- cập nhật sách
- cập nhật trạng thái phiếu

14. Yêu cầu SOLID

Dự án phải thể hiện được các nguyên tắc sau:

S — Single Responsibility Principle

Mỗi lớp chỉ nên có một trách nhiệm chính:

- View chỉ lo hiển thị
- Controller chỉ nhận sự kiện và điều phối
- Service xử lý nghiệp vụ
- Repository thao tác dữ liệu

O — Open/Closed Principle

Thiết kế để dễ mở rộng:

- có thể thêm loại báo cáo mới
- có thể thêm kiểu tìm kiếm mới
- không sửa quá nhiều mã cũ

L — Liskov Substitution Principle

Nếu có kế thừa:

- Admin có thể kế thừa Staff
- các lớp con không làm sai hành vi lớp cha

I — Interface Segregation Principle

Không tạo interface quá lớn.

Ví dụ:

- BookService
- ReaderService
- LoanService

  thay vì gom tất cả vào một interface khổng lồ

D — Dependency Inversion Principle

Controller phụ thuộc vào abstraction:

- BookController nên dùng BookService
- BookServiceImpl mới phụ thuộc BookRepository

15. Ví dụ interface và lớp triển khai

```
public interface BookService {
   List<Book> findAll();
   Book save(Book book);
   Book update(Book book);
   void delete(Long id);
   List<Book> searchByTitle(String keyword);
}
public class BookServiceImpl implements BookService {
   private final BookRepository bookRepository;

   public BookServiceImpl(BookRepository bookRepository) {
      this.bookRepository = bookRepository;
   }

   @Override
   public List<Book> findAll() {
      return bookRepository.findAll();
   }

   @Override
   public Book save(Book book) {
      // validate nghiệp vụ
      return bookRepository.save(book);
   }

   @Override
   public Book update(Book book) {
      return bookRepository.update(book);
   }

   @Override
   public void delete(Long id) {
      bookRepository.delete(id);
   }

   @Override
   public List<Book> searchByTitle(String keyword) {
      return bookRepository.searchByTitle(keyword);
   }
}
```

16. Màn hình giao diện đề xuất

16.1. Form đăng nhập

- username
- password
- nút login

16.2. Main Dashboard

- menu trái hoặc tab
- Books
- Readers
- Loans
- Returns
- Statistics

16.3. Form quản lý sách

- bảng danh sách sách
- form nhập
- nút Add / Update / Delete / Clear / Search

16.4. Form quản lý độc giả

- bảng danh sách độc giả
- tìm kiếm theo mã/tên

16.5. Form lập phiếu mượn

- chọn độc giả
- bảng chọn sách
- giỏ mượn
- nút xác nhận

16.6. Form trả sách

- nhập mã phiếu
- hiển thị danh sách sách đang mượn
- trả từng cuốn

16.7. Form thống kê

- tổng số sách
- tổng số độc giả
- phiếu chưa trả
- sách mượn nhiều

17. Cấu trúc Maven đề xuất

pom.xml cần có

- Java version
- dependency Hibernate
- dependency JPA API
- dependency MySQL
- logging

Ví dụ nhóm sinh viên cần tự cấu hình:

- persistence.xml hoặc Hibernate config
- database schema
- charset UTF-8

18. Yêu cầu nộp UML

Nhóm phải nộp tối thiểu:

18.1. Use Case Diagram

- đầy đủ actor
- đầy đủ use case chính
- có include/extend hợp lý

18.2. Activity Diagram

- ít nhất 3 activity
- có điều kiện rẽ nhánh
- có start/end rõ ràng

18.3. Sequence Diagram

- ít nhất 3 sequence
- thể hiện đúng MVC + Service + Repository

18.4. Class Diagram

Phải có:

- lớp entity
- lớp controller
- lớp service/interface
- lớp repository
- quan hệ kế thừa/phụ thuộc/kết hợp

- quan hệ giữa entity khớp với ERD

19. Yêu cầu nộp CSDL

Nhóm cần nộp:

- file .sql tạo database
- file insert dữ liệu mẫu
- mô tả ERD
- giải thích các khóa:
  - PK
  - FK
  - unique
  - not null

20. Phân công nhóm 2–3 sinh viên

Phương án nhóm 2 sinh viên

Sinh viên 1

- thiết kế database
- entity JPA
- repository
- chức năng quản lý sách, thể loại, tác giả

Sinh viên 2

- giao diện Swing
- controller
- chức năng độc giả, mượn – trả, thống kê
- tích hợp toàn hệ thống

Phương án nhóm 3 sinh viên

Sinh viên 1

- ERD, Entity, Repository

Sinh viên 2

- View + Controller cho sách, tác giả, thể loại, NXB

Sinh viên 3

- View + Controller cho độc giả, mượn – trả, thống kê, test tích hợp

21. Yêu cầu báo cáo

Báo cáo nên gồm:

1. Giới thiệu đề tài
2. Mục tiêu
3. Phân tích yêu cầu
4. Use Case Diagram
5. Activity Diagram
6. Sequence Diagram
7. Class Diagram
8. ERD và thiết kế database
9. Kiến trúc MVC + JPA
10. Mô tả SOLID đã áp dụng
11. Hình ảnh giao diện
12. Kết quả chạy thử
13. Khó khăn và hướng mở rộng

22. Hướng mở rộng

Nếu nhóm làm tốt có thể cộng điểm khi thêm:

- phân quyền Admin / Staff
- xuất Excel hoặc PDF
- biểu đồ thống kê
- tìm kiếm nâng cao

- kiểm tra trễ hạn
- tính tiền phạt trễ hạn
- upload ảnh bìa sách
- dark mode giao diện

23. Tiêu chí chấm điểm gợi ý

1. Phân tích và thiết kế: 25%

- Use Case đúng
- Activity rõ
- Sequence đúng luồng
- Class Diagram khớp dữ liệu

2. Thiết kế CSDL và JPA: 20%

- ERD hợp lý
- mapping JPA đúng
- quan hệ entity chuẩn

3. Lập trình chức năng: 30%

- chạy được
- CRUD đầy đủ
- mượn – trả đúng nghiệp vụ

4. Kiến trúc và SOLID: 15%

- tách lớp rõ ràng
- không nhồi code vào UI
- có interface/service/repository

5. Giao diện và trình bày: 10%

- dễ dùng
- bố cục hợp lý
- báo cáo sạch đẹp

24. Mô tả đầu bài ngắn gọn

Đề tài: Xây dựng hệ thống quản lý thư viện bằng Java Swing MVC + Hibernate/JPA + MySQL

Yêu cầu:

Nhóm 2–3 sinh viên xây dựng một ứng dụng desktop quản lý thư viện chạy trên IntelliJ IDEA bằng Maven. Hệ thống phải sử dụng Java Swing cho giao diện, Hibernate/JPA để kết nối MySQL và tổ chức mã nguồn theo mô hình MVC. Trong quá trình phát triển, nhóm cần áp dụng các nguyên tắc SOLID.

Chức năng tối thiểu:

- Đăng nhập
- Quản lý sách
- Quản lý tác giả
- Quản lý thể loại
- Quản lý nhà xuất bản
- Quản lý độc giả
- Lập phiếu mượn
- Trả sách
- Tìm kiếm
- Thống kê cơ bản

Hồ sơ nộp:

- source code Maven
- database .sql
- báo cáo PDF/Word
- UML gồm:
    - Use Case Diagram

    - Activity Diagram
    - Sequence Diagram
    - Class Diagram
- slide thuyết trình
- video demo ngắn

Lưu ý:

- Class Diagram phải phù hợp với thiết kế ERD
- Không viết toàn bộ logic trong form Swing
- Phải có entity, repository, service, controller rõ ràng
- Dự án phải chạy được với MySQL

Java Swing MVC + Hibernate/JPA + MySQL
Library Management System

Sinh viên trình bày có đủ:

- *báo cáo*
- *slide*
- *UML*
- *nộp source code*
- *demo cuối kỳ*

1. Cấu trúc nộp đồ án chuẩn IntelliJ Maven

```
LibraryManagementSystem/
├── .idea/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/library/
│   │   │       ├── AppLauncher.java
│   │   │       ├── config/
│   │   │       │   └── JpaUtil.java
│   │   │       ├── controller/
│   │   │       │   ├── AuthController.java
│   │   │       │   ├── BookController.java
│   │   │       │   ├── LoanController.java
│   │   │       │   ├── ReaderController.java
│   │   │       │   └── StatisticsController.java
│   │   │       ├── model/
│   │   │       │   ├── dto/
│   │   │       │   │   ├── DashboardStats.java
│   │   │       │   │   ├── LoanItemRequest.java
│   │   │       │   │   └── TopBorrowedBook.java
│   │   │       │   ├── entity/
│   │   │       │   │   ├── Author.java
│   │   │       │   │   ├── Book.java
│   │   │       │   │   ├── Category.java
│   │   │       │   │   ├── Loan.java
│   │   │       │   │   ├── LoanDetail.java
│   │   │       │   │   ├── Publisher.java
│   │   │       │   │   ├── Reader.java
│   │   │       │   │   └── Staff.java
│   │   │       │   └── enums/
│   │   │       │       ├── LoanStatus.java
│   │   │       │       ├── ReaderGender.java
│   │   │       │       ├── ReaderStatus.java
```

```
│   │   │   │   │   └── StaffRole.java
│   │   │   ├── repository/
│   │   │   │   ├── BookRepository.java
│   │   │   │   ├── CategoryRepository.java
│   │   │   │   ├── LoanRepository.java
│   │   │   │   ├── PublisherRepository.java
│   │   │   │   ├── ReaderRepository.java
│   │   │   │   ├── StaffRepository.java
│   │   │   │   ├── StatisticsRepository.java
│   │   │   │   └── impl/
│   │   │   │       ├── BookRepositoryImpl.java
│   │   │   │       ├── CategoryRepositoryImpl.java
│   │   │   │       ├── LoanRepositoryImpl.java
│   │   │   │       ├── PublisherRepositoryImpl.java
│   │   │   │       ├── ReaderRepositoryImpl.java
│   │   │   │       ├── StaffRepositoryImpl.java
│   │   │   │       └── StatisticsRepositoryImpl.java
│   │   │   ├── service/
│   │   │   │   ├── AuthService.java
│   │   │   │   ├── BookService.java
│   │   │   │   ├── LoanService.java
│   │   │   │   ├── ReaderService.java
│   │   │   │   ├── StatisticsService.java
│   │   │   │   └── impl/
│   │   │   │       ├── AuthServiceImpl.java
│   │   │   │       ├── BookServiceImpl.java
│   │   │   │       ├── LoanServiceImpl.java
│   │   │   │       ├── ReaderServiceImpl.java
│   │   │   │       └── StatisticsServiceImpl.java
│   │   │   └── view/
│   │   │       ├── auth/
│   │   │       │   └── LoginFrame.java
│   │   │       ├── book/
│   │   │       │   └── BookManagementPanel.java
│   │   │       ├── dashboard/
│   │   │       │   └── DashboardPanel.java
│   │   │       ├── loan/
│   │   │       │   └── LoanManagementPanel.java
│   │   │       ├── main/
│   │   │       │   └── MainFrame.java
│   │   │       ├── reader/
│   │   │       │   └── ReaderManagementPanel.java
│   │   │       ├── returnbook/
│   │   │       │   └── ReturnBookPanel.java
│   │   │       └── statistics/
│   │   │           └── StatisticsPanel.java
│   │   └── resources/
│   │       └── META-INF/
│   │           └── persistence.xml
│   └── test/
│       └── java/
├── database/
│   ├── 01_create_database.sql
│   ├── 02_create_tables.sql
│   └── 03_insert_sample_data.sql
├── uml/
```

```
│   ├── usecase_library.puml
│   ├── activity_create_loan.puml
│   ├── activity_return_book.puml
│   ├── sequence_add_book.puml
│   ├── sequence_create_loan.puml
│   ├── sequence_return_book.puml
│   ├── class_diagram_library.puml
│   └── package_diagram_library.puml
├── docs/
│   ├── Report.pdf
│   ├── Slide.pptx
│   └── Screenshots/
├── pom.xml
├── README.md
└── .gitignore
```

2. README.md mẫu chuẩn nộp đồ án

Nội dung sau vào file README.md.

Library Management System

1. Project Overview
Library Management System is a desktop application developed using Java Swing, MVC architecture, Hibernate/JPA, and MySQL.
The system supports book management, reader management, borrowing and returning books, authentication, dashboard, and statistics.

2. Technologies Used
- Java 17
- Java Swing
- Maven
- Hibernate ORM / JPA
- MySQL 8
- IntelliJ IDEA

3. Main Features
- Login with role-based access (ADMIN / STAFF)
- Book management
- Reader management
- Borrow book
- Return book
- Dashboard summary
- Statistics for top borrowed books
- MySQL database integration with Hibernate/JPA

4. Architecture
The project follows MVC + Service + Repository architecture.

- View: Java Swing forms and panels
- Controller: coordinates between UI and service
- Service: handles business logic
- Repository: database access using JPA
- Entity: persistent domain model

5. Project Structure
- `src/main/java`: application source code
- `src/main/resources`: configuration files
- `database`: MySQL scripts

- `uml`: PlantUML diagrams
- `docs`: report, slides, screenshots

6. Database Setup
Step 1: Create database
Run:
- `database/01_create_database.sql`
- `database/02_create_tables.sql`
- `database/03_insert_sample_data.sql`

Step 2: Update database config
Edit `src/main/resources/META-INF/persistence.xml`

Change:
- database username
- database password
- JDBC URL if needed

Example:
```xml
<property name="jakarta.persistence.jdbc.url"

value="jdbc:mysql://localhost:3306/library_management?useSSL=false&amp;serverTimezone=Asia/Ho_Chi_Minh&amp;allowPublicKeyRetrieval=true"/>
<property name="jakarta.persistence.jdbc.user" value="root"/>
<property name="jakarta.persistence.jdbc.password" value="YOUR_MYSQL_PASSWORD"/>
```

7. How to Run

1. Open project in IntelliJ IDEA
2. Wait for Maven dependencies to download
3. Run AppLauncher.java
4. Login using sample accounts below

8. Sample Accounts

Admin

- Username: admin
- Password: 123456

Staff

- Username: staff
- Password: 123456

9. UML Diagrams

The UML diagrams are stored in the uml/ folder:

- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Class Diagram
- Package Diagram

10. Screenshots

Add screenshots into docs/Screenshots/ and reference them here.

11. Team Members

- Student 1: Database + JPA Entities + Repository
- Student 2: Swing UI + Controller + CRUD
- Student 3: Loan/Return + Dashboard + Statistics + UML + Report

12. Role Distribution

ADMIN

- View dashboard
- Manage books
- Manage readers

- Borrow books
- Return books
- View statistics

STAFF

- View dashboard
- Manage books
- Manage readers
- Borrow books
- Return books

13. Future Improvements

- Password encryption with BCrypt
- Export Excel/PDF
- Advanced search and filter
- Overdue notification
- Staff management panel
- Audit log

14. Conclusion

This project demonstrates how to build a real desktop information system using Java Swing, MVC, Hibernate/JPA, and MySQL in a clean and extensible architecture.

---

3. `.gitignore` mẫu cho IntelliJ Maven

```gitignore
target/
.idea/
*.iml
out/
logs/
.DS_Store
```

4. Bộ PlantUML đầy đủ

4.1. Use Case Diagram

File: uml/usecase_library.puml

```
@startuml
left to right direction
skinparam actorStyle awesome

actor Staff
actor Admin

Admin --|> Staff

rectangle "Library Management System" {
  usecase "Login" as UC1
  usecase "View Dashboard" as UC2
  usecase "Manage Books" as UC3
  usecase "Manage Readers" as UC4
  usecase "Create Loan" as UC5
  usecase "Return Book" as UC6
  usecase "Search Loan" as UC7
  usecase "View Statistics" as UC8
  usecase "Logout" as UC9
  usecase "Check Available Quantity" as UC10
  usecase "Update Book Stock" as UC11
}
```

```
Staff --> UC1
Staff --> UC2
Staff --> UC3
Staff --> UC4
Staff --> UC5
Staff --> UC6
Staff --> UC7
Staff --> UC9

Admin --> UC8

UC5 .> UC10 : <<include>>
UC5 .> UC11 : <<include>>
UC6 .> UC11 : <<include>>
@enduml
```

4.2. Activity Diagram: Create Loan

File: uml/activity_create_loan.puml

```
@startuml
start

:Open Loan Management Panel;
:Enter loan code, reader, staff, dates;
:Add books into loan cart;

if (Loan data valid?) then (Yes)
  :Check reader exists;
  :Check staff exists;
  :Check reader status = ACTIVE;
  :Check each book quantity;
  if (All books available?) then (Yes)
    :Create Loan;
    :Create LoanDetail records;
    :Decrease availableQuantity of books;
    :Commit transaction;
    :Show success message;
  else (No)
    :Show insufficient stock error;
  endif
else (No)
  :Show validation error;
endif

stop
@enduml
```

4.3. Activity Diagram: Return Book

File: uml/activity_return_book.puml

```
@startuml
start

:Open Return Book Panel;
:Enter loan code;
:Search loan;

if (Loan found?) then (Yes)
```

```
 :Display loan details;
 :Select loan detail row;
 :Enter return quantity and return date;

 if (Return quantity valid?) then (Yes)
  :Update returnedQuantity;
  :Update returnDate;
  :Increase availableQuantity of book;
  :Recalculate loan status;
  :Commit transaction;
  :Show success message;
 else (No)
  :Show invalid quantity error;
 endif

else (No)
 :Show loan not found error;
endif

stop
@enduml
```

4.4. Sequence Diagram: Add Book

File: uml/sequence_add_book.puml

```
@startuml
actor Staff
participant "BookManagementPanel" as View
participant "BookController" as Controller
participant "BookService" as Service
participant "BookRepository" as Repo
database "MySQL" as DB

Staff -> View : Enter book information
Staff -> View : Click Add
View -> Controller : createBook(...)
Controller -> Service : createBook(...)
Service -> Repo : existsByIsbn(isbn)
Repo -> DB : SELECT COUNT(*)
DB --> Repo : result
Repo --> Service : exists / not exists

alt ISBN not exists
 Service -> Repo : save(book)
 Repo -> DB : INSERT INTO books ...
 DB --> Repo : saved
 Repo --> Service : Book
 Service --> Controller : Book
 Controller --> View : Book
 View --> Staff : Show success message
else ISBN exists
 Service --> Controller : throw error
 Controller --> View : error message
 View --> Staff : Show validation error
end
@enduml
```

4.5. Sequence Diagram: Create Loan

File: uml/sequence_create_loan.puml

```
@startuml
actor Staff
participant "LoanManagementPanel" as View
participant "LoanController" as Controller
participant "LoanService" as Service
database "MySQL / JPA" as DB

Staff -> View : Enter loan info and items
Staff -> View : Click Create Loan
View -> Controller : createLoan(...)
Controller -> Service : createLoan(...)

Service -> DB : Begin transaction
Service -> DB : Find Reader
Service -> DB : Find Staff
Service -> DB : Validate Reader ACTIVE

loop For each loan item
  Service -> DB : Find Book
  Service -> DB : Check availableQuantity
end

alt Valid and sufficient stock
  Service -> DB : Persist Loan
  loop For each loan item
    Service -> DB : Persist LoanDetail
    Service -> DB : Update Book.availableQuantity
  end
  Service -> DB : Commit
  Service --> Controller : Loan
  Controller --> View : Loan created
  View --> Staff : Show success
else Validation failed
  Service -> DB : Rollback
  Service --> Controller : throw error
  Controller --> View : error
  View --> Staff : Show error
end
@enduml
```

4.6. Sequence Diagram: Return Book

File: uml/sequence_return_book.puml

```
@startuml
actor Staff
participant "ReturnBookPanel" as View
participant "LoanController" as Controller
participant "LoanService" as Service
database "MySQL / JPA" as DB

Staff -> View : Enter loan code
View -> Controller : getLoanByCode(code)
Controller -> Service : getLoanByCode(code)
Service -> DB : Query loan + details
DB --> Service : Loan
Service --> Controller : Loan
```

```
Controller --> View : Loan details

Staff -> View : Select loan detail and return qty
Staff -> View : Click Return
View -> Controller : returnBook(loanDetailId, qty, date)
Controller -> Service : returnBook(...)

Service -> DB : Begin transaction
Service -> DB : Find LoanDetail
Service -> DB : Validate remaining quantity
Service -> DB : Update LoanDetail.returnedQuantity
Service -> DB : Update LoanDetail.returnDate
Service -> DB : Update Book.availableQuantity
Service -> DB : Recalculate Loan.status
Service -> DB : Commit

Service --> Controller : Updated Loan
Controller --> View : Updated Loan
View --> Staff : Show success
@enduml
```

4.7. Class Diagram

File: uml/class_diagram_library.puml

```
@startuml
skinparam classAttributeIconSize 0

package "model.enums" {
 enum ReaderGender {
  MALE
  FEMALE
  OTHER
 }

 enum ReaderStatus {
  ACTIVE
  INACTIVE
  BLOCKED
 }

 enum StaffRole {
  ADMIN
  STAFF
 }

 enum LoanStatus {
  BORROWING
  PARTIAL_RETURNED
  RETURNED
  OVERDUE
  CANCELLED
 }
}

package "model.entity" {
 class Category {
  - Long id
  - String name
```

```
  - String description
}

class Publisher {
  - Long id
  - String name
  - String address
  - String phone
}

class Author {
  - Long id
  - String fullName
  - String email
  - String bio
}

class Book {
  - Long id
  - String isbn
  - String title
  - Integer publishYear
  - BigDecimal price
  - Integer quantity
  - Integer availableQuantity
}

class Reader {
  - Long id
  - String readerCode
  - String fullName
  - ReaderGender gender
  - String phone
  - String email
  - String address
  - ReaderStatus status
}

class Staff {
  - Long id
  - String staffCode
  - String fullName
  - String username
  - String password
  - StaffRole role
}

class Loan {
  - Long id
  - String loanCode
  - LocalDate loanDate
  - LocalDate dueDate
  - LoanStatus status
}

class LoanDetail {
```

```
  - Long id
  - Integer quantity
  - Integer returnedQuantity
  - LocalDate returnDate
  - String note
 }
}

package "repository" {
 interface BookRepository
 interface ReaderRepository
 interface LoanRepository
 interface StaffRepository
 interface StatisticsRepository
}

package "service" {
 interface AuthService
 interface BookService
 interface ReaderService
 interface LoanService
 interface StatisticsService
}

package "controller" {
 class AuthController
 class BookController
 class ReaderController
 class LoanController
 class StatisticsController
}

package "view" {
 class LoginFrame
 class MainFrame
 class DashboardPanel
 class BookManagementPanel
 class ReaderManagementPanel
 class LoanManagementPanel
 class ReturnBookPanel
 class StatisticsPanel
}

Category "1" <-- "0..*" Book : category
Publisher "1" <-- "0..*" Book : publisher
Book "0..*" -- "0..*" Author : authors
Reader "1" <-- "0..*" Loan : reader
Staff "1" <-- "0..*" Loan : staff
Loan "1" <-- "1..*" LoanDetail : loan
Book "1" <-- "0..*" LoanDetail : book

AuthController --> AuthService
BookController --> BookService
ReaderController --> ReaderService
LoanController --> LoanService
StatisticsController --> StatisticsService
```

```
BookService --> BookRepository
ReaderService --> ReaderRepository
LoanService --> LoanRepository
AuthService --> StaffRepository
StatisticsService --> StatisticsRepository

LoginFrame --> AuthController
MainFrame --> DashboardPanel
MainFrame --> BookManagementPanel
MainFrame --> ReaderManagementPanel
MainFrame --> LoanManagementPanel
MainFrame --> ReturnBookPanel
MainFrame --> StatisticsPanel
@enduml
```

4.8. Package Diagram

File: uml/package_diagram_library.puml

```
@startuml
package "view" {
}

package "controller" {
}

package "service" {
}

package "repository" {
}

package "model.entity" {
}

package "model.dto" {
}

package "model.enums" {
}

package "config" {
}

view --> controller
controller --> service
service --> repository
repository --> "model.entity"
service --> "model.entity"
service --> "model.dto"
"model.entity" --> "model.enums"
repository --> config
@enduml
```

5. Cách mô tả phần UML trong báo cáo

Bạn có thể ghi ngắn gọn như sau:

Use Case Diagram

Mô tả các chức năng chính mà hệ thống cung cấp cho hai tác nhân là STAFF và ADMIN. ADMIN kế thừa toàn bộ quyền của STAFF và được phép xem thống kê.

Activity Diagram

Mô tả chi tiết luồng xử lý của hai nghiệp vụ cốt lõi:

- lập phiếu mượn
- trả sách

Sequence Diagram

Thể hiện luồng tương tác giữa:

- giao diện
- controller
- service
- repository
- database

theo đúng kiến trúc MVC + Service + Repository.

Class Diagram

Phản ánh cấu trúc tĩnh của hệ thống, gồm:

- entity classes
- service interfaces
- repositories
- controllers
- views

Đặc biệt, các entity classes khớp với ERD và các bảng trong MySQL.

6. Gợi ý đóng gói bài nộp cho giảng viên

Nên nén thư mục dự án theo kiểu:

22110301_22110302_22110303_LibraryManagementSystem.zip

Bên trong gồm:

- source code IntelliJ Maven
- thư mục database
- thư mục uml
- báo cáo PDF
- slide trình bày
- ảnh chụp màn hình
- video demo nếu có

7. Checklist trước khi nộp

- Project mở được bằng IntelliJ
- mvn clean compile chạy được
- import DB thành công
- AppLauncher chạy được
- login được bằng tài khoản mẫu
- CRUD sách hoạt động
- CRUD độc giả hoạt động
- tạo phiếu mượn hoạt động
- trả sách hoạt động
- dashboard hiển thị số liệu
- admin thấy thống kê
- staff không thấy thống kê
- thư mục uml/ có đầy đủ file .puml
- README có hướng dẫn chạy

8. Tên file UML khuyến nghị

Để đẹp và chuyên nghiệp, nhóm sinh viên nên nộp đúng các file tên như sau:

- usecase_library.puml

- activity_create_loan.puml
- activity_return_book.puml
- sequence_add_book.puml
- sequence_create_loan.puml
- sequence_return_book.puml
- class_diagram_library.puml
- package_diagram_library.puml

9. Nhận xét chuyên môn

Bộ này đã đủ tốt để:

- nộp đồ án
- trình bày báo cáo
- minh họa rõ kiến trúc MVC
- chứng minh dùng Hibernate/JPA đúng cách
- thể hiện thiết kế UML khớp code và database

Một số nội dung mẫu tham khảo:

- pom.xml hoàn chỉnh
- bộ script tách sẵn:
  - 01_create_database.sql
  - 02_create_tables.sql
  - 03_insert_sample_data.sql
- README.md bản tiếng Việt chuẩn nộp giảng viên

Bộ version thực dụng cho Maven project: Hibernate ORM 7.3.0.Final, Jakarta Persistence API 3.2.0, MySQL Connector/J 9.6.0, SLF4J Simple 2.0.17, Maven Compiler Plugin 3.15.0, và Exec Maven Plugin 3.5.0. Apache Maven hiện khuyến nghị Maven 3.9.14. (Maven Repository)

1. pom.xml hoàn chỉnh

Lưu tại:

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="
        http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>library-management-system</artifactId>
  <version>1.0.0</version>
  <name>Library Management System</name>
  <description>Java Swing MVC + Hibernate/JPA + MySQL</description>
  <packaging>jar</packaging>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
    <main.class>com.example.library.AppLauncher</main.class>

    <hibernate.version>7.3.0.Final</hibernate.version>
    <jakarta.persistence.version>3.2.0</jakarta.persistence.version>
    <mysql.connector.version>9.6.0</mysql.connector.version>
    <slf4j.version>2.0.17</slf4j.version>
    <maven.compiler.plugin.version>3.15.0</maven.compiler.plugin.version>
```

```
    <exec.maven.plugin.version>3.5.0</exec.maven.plugin.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>${hibernate.version}</version>
    </dependency>

    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>${jakarta.persistence.version}</version>
    </dependency>

    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>${mysql.connector.version}</version>
    </dependency>

    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>${slf4j.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>${maven.compiler.plugin.version}</version>
        <configuration>
          <release>${maven.compiler.release}</release>
        </configuration>
      </plugin>

      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>${exec.maven.plugin.version}</version>
        <configuration>
          <mainClass>${main.class}</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

2. database/01_create_database.sql

Lưu tại:

database/01_create_database.sql

```
DROP DATABASE IF EXISTS library_management;
```

```sql
CREATE DATABASE library_management
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE library_management;
```

3. database/02_create_tables.sql

Lưu tại:
database/02_create_tables.sql

```sql
USE library_management;

-- ==========================================
-- 1. categories
-- ==========================================
CREATE TABLE categories (
  id BIGINT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  description VARCHAR(255),
  PRIMARY KEY (id),
  CONSTRAINT uk_categories_name UNIQUE (name)
) ENGINE=InnoDB;

-- ==========================================
-- 2. publishers
-- ==========================================
CREATE TABLE publishers (
  id BIGINT NOT NULL AUTO_INCREMENT,
  name VARCHAR(150) NOT NULL,
  address VARCHAR(255),
  phone VARCHAR(20),
  PRIMARY KEY (id),
  CONSTRAINT uk_publishers_name UNIQUE (name)
) ENGINE=InnoDB;

-- ==========================================
-- 3. authors
-- ==========================================
CREATE TABLE authors (
  id BIGINT NOT NULL AUTO_INCREMENT,
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(150),
  bio TEXT,
  PRIMARY KEY (id),
  CONSTRAINT uk_authors_email UNIQUE (email)
) ENGINE=InnoDB;

-- ==========================================
-- 4. books
-- ==========================================
CREATE TABLE books (
  id BIGINT NOT NULL AUTO_INCREMENT,
  isbn VARCHAR(20) NOT NULL,
  title VARCHAR(255) NOT NULL,
  publish_year INT,
  price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
```

```
  quantity INT NOT NULL DEFAULT 0,
  available_quantity INT NOT NULL DEFAULT 0,
  category_id BIGINT NOT NULL,
  publisher_id BIGINT NOT NULL,
  PRIMARY KEY (id),
  CONSTRAINT uk_books_isbn UNIQUE (isbn),
  CONSTRAINT ck_books_quantity CHECK (quantity >= 0),
  CONSTRAINT ck_books_available_quantity CHECK (available_quantity >= 0),
  CONSTRAINT ck_books_publish_year CHECK (publish_year IS NULL OR publish_year >= 1000),
  CONSTRAINT fk_books_category
    FOREIGN KEY (category_id) REFERENCES categories(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT fk_books_publisher
    FOREIGN KEY (publisher_id) REFERENCES publishers(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE INDEX idx_books_title ON books(title);
CREATE INDEX idx_books_category_id ON books(category_id);
CREATE INDEX idx_books_publisher_id ON books(publisher_id);

-- ==========================================
-- 5. book_authors
-- ==========================================
CREATE TABLE book_authors (
  book_id BIGINT NOT NULL,
  author_id BIGINT NOT NULL,
  PRIMARY KEY (book_id, author_id),
  CONSTRAINT fk_book_authors_book
    FOREIGN KEY (book_id) REFERENCES books(id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_book_authors_author
    FOREIGN KEY (author_id) REFERENCES authors(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE INDEX idx_book_authors_author_id ON book_authors(author_id);

-- ==========================================
-- 6. readers
-- ==========================================
CREATE TABLE readers (
  id BIGINT NOT NULL AUTO_INCREMENT,
  reader_code VARCHAR(20) NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  gender VARCHAR(10),
  phone VARCHAR(20),
  email VARCHAR(150),
  address VARCHAR(255),
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (id),
  CONSTRAINT uk_readers_reader_code UNIQUE (reader_code),
```

```
  CONSTRAINT uk_readers_email UNIQUE (email),
  CONSTRAINT ck_readers_gender CHECK (gender IN ('MALE', 'FEMALE', 'OTHER') OR gender IS
NULL),
  CONSTRAINT ck_readers_status CHECK (status IN ('ACTIVE', 'INACTIVE', 'BLOCKED'))
) ENGINE=InnoDB;

CREATE INDEX idx_readers_full_name ON readers(full_name);
CREATE INDEX idx_readers_phone ON readers(phone);

-- ==========================================
-- 7. staffs
-- ==========================================
CREATE TABLE staffs (
  id BIGINT NOT NULL AUTO_INCREMENT,
  staff_code VARCHAR(20) NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'STAFF',
  PRIMARY KEY (id),
  CONSTRAINT uk_staffs_staff_code UNIQUE (staff_code),
  CONSTRAINT uk_staffs_username UNIQUE (username),
  CONSTRAINT ck_staffs_role CHECK (role IN ('ADMIN', 'STAFF'))
) ENGINE=InnoDB;

CREATE INDEX idx_staffs_full_name ON staffs(full_name);

-- ==========================================
-- 8. loans
-- ==========================================
CREATE TABLE loans (
  id BIGINT NOT NULL AUTO_INCREMENT,
  loan_code VARCHAR(20) NOT NULL,
  reader_id BIGINT NOT NULL,
  staff_id BIGINT NOT NULL,
  loan_date DATE NOT NULL,
  due_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'BORROWING',
  PRIMARY KEY (id),
  CONSTRAINT uk_loans_loan_code UNIQUE (loan_code),
  CONSTRAINT ck_loans_status CHECK (
    status IN ('BORROWING', 'PARTIAL_RETURNED', 'RETURNED', 'OVERDUE', 'CANCELLED')
  ),
  CONSTRAINT ck_loans_due_date CHECK (due_date >= loan_date),
  CONSTRAINT fk_loans_reader
    FOREIGN KEY (reader_id) REFERENCES readers(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT fk_loans_staff
    FOREIGN KEY (staff_id) REFERENCES staffs(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE INDEX idx_loans_reader_id ON loans(reader_id);
CREATE INDEX idx_loans_staff_id ON loans(staff_id);
```

```sql
CREATE INDEX idx_loans_loan_date ON loans(loan_date);
CREATE INDEX idx_loans_status ON loans(status);

-- ==========================================
-- 9. loan_details
-- ==========================================
CREATE TABLE loan_details (
  id BIGINT NOT NULL AUTO_INCREMENT,
  loan_id BIGINT NOT NULL,
  book_id BIGINT NOT NULL,
  quantity INT NOT NULL DEFAULT 1,
  returned_quantity INT NOT NULL DEFAULT 0,
  return_date DATE NULL,
  note VARCHAR(255),
  PRIMARY KEY (id),
  CONSTRAINT ck_loan_details_quantity CHECK (quantity > 0),
  CONSTRAINT ck_loan_details_returned_quantity CHECK (returned_quantity >= 0),
  CONSTRAINT ck_loan_details_returned_le_quantity CHECK (returned_quantity <= quantity),
  CONSTRAINT fk_loan_details_loan
    FOREIGN KEY (loan_id) REFERENCES loans(id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_loan_details_book
    FOREIGN KEY (book_id) REFERENCES books(id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE INDEX idx_loan_details_loan_id ON loan_details(loan_id);
CREATE INDEX idx_loan_details_book_id ON loan_details(book_id);
```

4. database/03_insert_sample_data.sql

Lưu tại:

database/03_insert_sample_data.sql

```sql
USE library_management;

-- ==========================================
-- SAMPLE DATA: categories
-- ==========================================
INSERT INTO categories(name, description) VALUES
('Programming', 'Books about programming'),
('Database', 'Books about databases'),
('Networking', 'Books about networking'),
('Soft Skills', 'Books about communication and teamwork');

-- ==========================================
-- SAMPLE DATA: publishers
-- ==========================================
INSERT INTO publishers(name, address, phone) VALUES
('O''Reilly Media', 'USA', '111-222-333'),
('Pearson', 'UK', '222-333-444'),
('Springer', 'Germany', '333-444-555');

-- ==========================================
-- SAMPLE DATA: authors
-- ==========================================
```

```
INSERT INTO authors(full_name, email, bio) VALUES
('Joshua Bloch', 'joshua@example.com', 'Java expert'),
('Martin Fowler', 'martin@example.com', 'Software architecture expert'),
('Thomas H. Cormen', 'cormen@example.com', 'Algorithm author'),
('Robert C. Martin', 'unclebob@example.com', 'Clean Code author');

-- ==========================================
-- SAMPLE DATA: books
-- ==========================================
INSERT INTO books(isbn, title, publish_year, price, quantity, available_quantity, category_id,
publisher_id) VALUES
('9780134685991', 'Effective Java', 2018, 450000, 10, 10, 1, 2),
('9780132350884', 'Clean Code', 2008, 420000, 8, 8, 1, 2),
('9780201633610', 'Design Patterns Explained', 2002, 390000, 5, 5, 1, 1),
('9780131103627', 'The C Programming Language', 1988, 350000, 6, 6, 1, 2),
('9780321573513', 'Database Systems', 2010, 500000, 7, 7, 2, 3);

-- ==========================================
-- SAMPLE DATA: book_authors
-- ==========================================
INSERT INTO book_authors(book_id, author_id) VALUES
(1, 1),
(2, 4),
(5, 2);

-- ==========================================
-- SAMPLE DATA: readers
-- ==========================================
INSERT INTO readers(reader_code, full_name, gender, phone, email, address, status) VALUES
('R001', 'Nguyen Van A', 'MALE', '0901000001', 'a.reader@example.com', 'Ho Chi Minh City', 'ACTIVE'),
('R002', 'Tran Thi B', 'FEMALE', '0901000002', 'b.reader@example.com', 'Da Nang', 'ACTIVE'),
('R003', 'Le Van C', 'MALE', '0901000003', 'c.reader@example.com', 'Ha Noi', 'BLOCKED');

-- ==========================================
-- SAMPLE DATA: staffs
-- ==========================================
INSERT INTO staffs(staff_code, full_name, username, password, role) VALUES
('S001', 'Admin User', 'admin', '123456', 'ADMIN'),
('S002', 'Staff User', 'staff', '123456', 'STAFF');

-- ==========================================
-- SAMPLE DATA: loans
-- ==========================================
INSERT INTO loans(loan_code, reader_id, staff_id, loan_date, due_date, status) VALUES
('L001', 1, 2, '2026-03-01', '2026-03-10', 'BORROWING'),
('L002', 2, 2, '2026-03-05', '2026-03-15', 'PARTIAL_RETURNED');

-- ==========================================
-- SAMPLE DATA: loan_details
-- ==========================================
INSERT INTO loan_details(loan_id, book_id, quantity, returned_quantity, return_date, note) VALUES
(1, 1, 1, 0, NULL, 'Borrowing Effective Java'),
(1, 2, 1, 0, NULL, 'Borrowing Clean Code'),
(2, 5, 2, 1, '2026-03-08', 'Returned 1 copy');
```

5. README.md bản chuẩn nộp giảng viên

Lưu tại:

README.md

HỆ THỐNG QUẢN LÝ THƯ VIỆN
Java Swing MVC + Hibernate/JPA + MySQL

---

1. Giới thiệu đề tài
Đây là dự án xây dựng phần mềm quản lý thư viện chạy trên desktop, được phát triển bằng Java Swing, sử dụng mô hình MVC, Hibernate/JPA để kết nối cơ sở dữ liệu MySQL.

Hệ thống hỗ trợ các chức năng chính:
- Đăng nhập hệ thống
- Quản lý sách
- Quản lý độc giả
- Lập phiếu mượn sách
- Trả sách
- Dashboard tổng quan
- Thống kê hệ thống
- Phân quyền ADMIN / STAFF

---

2. Công nghệ sử dụng
- Java 17
- Java Swing
- Maven
- Hibernate ORM / JPA
- MySQL 8+
- IntelliJ IDEA

---

3. Kiến trúc hệ thống
Dự án được tổ chức theo kiến trúc:

- **View**: giao diện Java Swing
- **Controller**: tiếp nhận thao tác từ giao diện
- **Service**: xử lý nghiệp vụ
- **Repository**: thao tác dữ liệu qua JPA
- **Entity**: ánh xạ đối tượng với bảng trong cơ sở dữ liệu

Luồng chính:
`View -> Controller -> Service -> Repository -> Database`

---

4. Chức năng chính

4.1. Đăng nhập
- Đăng nhập bằng tài khoản nhân viên trong bảng `staffs`
- Phân quyền ADMIN / STAFF

4.2. Quản lý sách
- Thêm sách
- Sửa sách
- Xóa sách

- Tìm kiếm sách
- Gán thể loại và nhà xuất bản

4.3. Quản lý độc giả
- Thêm độc giả
- Sửa độc giả
- Xóa độc giả
- Tìm kiếm độc giả

4.4. Quản lý mượn sách
- Tạo phiếu mượn
- Mỗi phiếu có nhiều dòng chi tiết
- Kiểm tra số lượng sách khả dụng
- Tự động giảm tồn kho khi mượn

4.5. Quản lý trả sách
- Tìm phiếu mượn
- Chọn dòng sách cần trả
- Cập nhật số lượng đã trả
- Tăng lại tồn kho sách
- Tự động cập nhật trạng thái phiếu mượn

4.6. Dashboard
- Tổng số sách
- Tổng số độc giả
- Số phiếu đang mở
- Số phiếu quá hạn

4.7. Thống kê
- Chỉ ADMIN được xem
- Top sách được mượn nhiều

---

5. Cấu trúc thư mục dự án

- `src/main/java`: mã nguồn chính
- `src/main/resources`: file cấu hình `persistence.xml`
- `database`: các script MySQL
- `uml`: file PlantUML
- `docs`: báo cáo, slide, ảnh chụp màn hình

---

6. Hướng dẫn tạo cơ sở dữ liệu

Bước 1: Tạo database
Chạy lần lượt 3 file trong thư mục `database`:

1. `01_create_database.sql`
2. `02_create_tables.sql`
3. `03_insert_sample_data.sql`

Bước 2: Cấu hình lại `persistence.xml`
Mở file:

`src/main/resources/META-INF/persistence.xml`

và chỉnh lại các thông số:
- username MySQL
- password MySQL
- JDBC URL nếu cần

Ví dụ:
```xml
<property name="jakarta.persistence.jdbc.url"

value="jdbc:mysql://localhost:3306/library_management?useSSL=false&amp;serverTimezone=Asia/Ho_Chi_Minh&amp;allowPublicKeyRetrieval=true"/>
<property name="jakarta.persistence.jdbc.user" value="root"/>
<property name="jakarta.persistence.jdbc.password" value="YOUR_MYSQL_PASSWORD"/>
```

7. Hướng dẫn chạy project

Cách 1: Chạy bằng IntelliJ IDEA

1. Mở project trong IntelliJ IDEA
2. Chờ Maven tải dependencies
3. Chạy file AppLauncher.java

Cách 2: Chạy bằng Maven

Mở terminal tại thư mục project và chạy:
mvn clean compile
mvn exec:java

8. Tài khoản mẫu

ADMIN

- Username: admin
- Password: 12345

STAFF

- Username: staff
- Password: 12345

9. Phân quyền

ADMIN

- Xem dashboard
- Quản lý sách
- Quản lý độc giả
- Lập phiếu mượn
- Trả sách
- Xem thống kê

STAFF

- Xem dashboard
- Quản lý sách
- Quản lý độc giả
- Lập phiếu mượn
- Trả sách

10. UML

Các sơ đồ UML được lưu trong thư mục uml/, gồm:

- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Class Diagram
- Package Diagram

11. Thành viên nhóm đề xuất

Sinh viên 1

- Thiết kế database
- Viết Entity classes

- Viết Repository

Sinh viên 2

- Viết giao diện Swing
- Viết Controller
- Viết CRUD Book / Reader

Sinh viên 3

- Viết nghiệp vụ Loan / Return
- Viết Dashboard / Statistics
- Hoàn thiện UML, báo cáo, slide

12. Các điểm nổi bật của đồ án

- Áp dụng mô hình MVC rõ ràng
- Dùng Hibernate/JPA thay cho JDBC thuần
- Có transaction cho nghiệp vụ mượn / trả sách
- Có phân quyền đăng nhập
- Có dashboard và thống kê
- Class Diagram khớp với cơ sở dữ liệu MySQL

13. Hướng phát triển thêm

- Mã hóa password bằng BCrypt
- Export Excel / PDF
- Thông báo sách quá hạn
- Quản lý nhân viên
- Quản lý tác giả, thể loại, nhà xuất bản bằng giao diện riêng
- Nhật ký thao tác hệ thống

14. Kết luận

Dự án minh họa cách xây dựng một hệ thống thông tin desktop tương đối hoàn chỉnh bằng Java Swing, Hibernate/JPA và MySQL, với kiến trúc rõ ràng, dễ mở rộng và phù hợp cho đồ án môn học nhóm 2–3 sinh viên.

---

6. `persistence.xml` để chạy đúng với pom.xml

Đây là bản phù hợp với `pom.xml` ở trên:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         version="3.0"
         xsi:schemaLocation="
           https://jakarta.ee/xml/ns/persistence
           https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

  <persistence-unit name="libraryPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

    <class>com.example.library.model.entity.Category</class>
    <class>com.example.library.model.entity.Publisher</class>
    <class>com.example.library.model.entity.Author</class>
    <class>com.example.library.model.entity.Book</class>
    <class>com.example.library.model.entity.Reader</class>
    <class>com.example.library.model.entity.Staff</class>
    <class>com.example.library.model.entity.Loan</class>
    <class>com.example.library.model.entity.LoanDetail</class>

    <properties>
      <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
      <property name="jakarta.persistence.jdbc.url"
```

```
value="jdbc:mysql://localhost:3306/library_management?useSSL=false&amp;serverTimezone=Asia/
Ho_Chi_Minh&amp;allowPublicKeyRetrieval=true"/>
        <property name="jakarta.persistence.jdbc.user" value="root"/>
        <property name="jakarta.persistence.jdbc.password" value="YOUR_MYSQL_PASSWORD"/>

        <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/>
        <property name="hibernate.hbm2ddl.auto" value="validate"/>
        <property name="hibernate.show_sql" value="true"/>
        <property name="hibernate.format_sql" value="true"/>
        <property name="hibernate.highlight_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

7. Cấu trúc dự án khi nộp

Nhóm nên nộp dạng:

22110301_22110302_22110303_LibraryManagementSystem.zip

Bên trong nên có:

- source code đầy đủ
- thư mục database
- thư mục uml
- README.md
- báo cáo PDF
- slide PowerPoint
- ảnh chụp giao diện
- video demo nếu giảng viên yêu cầu

8. Checklist trước khi nộp

- Import database thành công
- Chỉnh đúng password trong persistence.xml
- mvn clean compile chạy không lỗi
- AppLauncher mở được
- Login được bằng admin / 12345
- Login được bằng staff / 12345
- CRUD sách chạy được
- CRUD độc giả chạy được
- Mượn sách chạy được
- Trả sách chạy được
- Dashboard chạy được
- ADMIN thấy menu thống kê
- STAFF không thấy menu thống kê
- Thư mục uml/ có đủ file .puml
- README ghi rõ cách chạy